DESARROLLO DE APLICACIONES CON ANGULAR

Angular

ANGULAR

- Animaciones
  - Permiten insertar animaciones CSS en las aplicaciones.

src/app/app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';

@NgModule({
  imports: [
    BrowserModule,
    BrowserAnimationsModule
  ],
  declarations: [ ],
  bootstrap: [ ]
})
export class AppModule { }
```

https://angular.io/guide/animations

ANGULAR

- Animaciones
  - Permiten insertar animaciones CSS en las aplicaciones.

src/app/app.component.ts

```
import { Component, HostBinding } from '@angular/core';
import {
  trigger,
  state,
  style,
  animate,
  transition,
  // ...
} from '@angular/animations';
```

https://angular.io/guide/animations

ANGULAR

- Animaciones
  - Se definen dentro del decorador **@Component**, con el nombre **animations** (array).

src/app/app.component.ts

```
@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  styleUrls: ['app.component.css'],
  animations: [
    // animation triggers go here
  ]
})
```

ANGULAR

- Animaciones
  - Construcción de la animación:
    - El elemento trigger tiene asignado un nombre y un array de estados.

```
animations: [
   trigger('nombreTrigger', [])
]
```

    - Se asigna el trigger al elemento html con @ junto con la variable que determina el estado.

```
<div [@nombreTrigger]="variableDeterminteEstado">
  <h1>Contenido</h1>
</div>
```

ANGULAR

- Animaciones
  - El **trigger** se compone estados y de transiciones.
  - Los estados se componen de nombre y de una sucesión de estilos css.
  - Las transiciones marcan la manera en que se van a realizar los cambios de estado.
    - estado_origen => estado_destino
      - estado1 => estado2
      - void => * →La primera transición.
      - *=>void → La última transición.
      - *=>estado
      - *=>*
      - estado1 <=>estado2

ANGULAR

- Animaciones
  - Alias de transiciones:
    - ‘:enter’ → void => *
    - ’:leave’ → *=>void
  - La transición :enter se ejecuta cuando cualquier vista *ngIf o *ngFor se coloca en la página, y :leave se ejecuta cuando esas vistas se eliminan de la página.

ANGULAR

▪Animaciones

```
<div [@nombreTrigger]="variableDeterminteEstado">
   <h1>Contenido</h1>
</div>
```

```
export class PresentadorComponent implements OnInit {
 variableDeterminanteEstado = 'open';

 conmutar() {
  if (this.variableDeterminanteEstado == 'open') {
   this.variableDeterminanteEstado = 'closed';
  } else {
   this.variableDeterminanteEstado = 'open';
  }

 }
```

```
animations: [
  trigger('nombreTrigger', [
    state('open', style({
      height: '200px',
      backgroundColor: 'yellow'
    })),
    state('closed', style({
      height: '100px',
      backgroundColor: 'blue'
    })),
    transition('open => closed', [
      animate('1s')
    ]),
    transition('closed => open', [
      animate('0.5s')
    ]),
  ])
]
```

ANGULAR

- Animaciones. KeyFrames.
  - Se puede detallar una animación a partir de keyframes.

```
transition('void=>*', [
    animate(10000, keyframes([
      style({ opacity: 0, transform: 'translate(0)', offset: 0 }),
      style({ opacity: 0.25, transform: 'translate(-100%)', offset: 0.25 }),
      style({ opacity: 0.5, transform: 'translate(100%)', offset: 0.5 }),
      style({ opacity: 0.75, transform: 'translate(-100%)', offset: 0.75 }),
      style({ opacity: 1, transform: 'translate(0)', offset: 1 })
    ])
  )])
```

ANGULAR

- Animaciones. Complex Sequences.
  - Las funciones que controlan las animaciones complejas son las siguientes:

FUNCTIONS	DETAILS
`query()`	Finds one or more inner HTML elements.
`stagger()`	Applies a cascading delay to animations for multiple elements.
`group()`	Runs multiple animation steps in parallel.
`sequence()`	Runs animation steps one after another.

ANGULAR

- Animaciones. Complex Sequences.
  - Ejemplo de query+stagger

```
<button (click)="agregar()">Agregar</button><button
(click)="eliminar()">Eliminar</button>
<main *ngFor="let item of items" [@trigger]="numeroItems">
    <div class="item efecto" >
        {{item}}
    </div>
</main>
```

ANGULAR

- Animaciones. Complex Sequences.
  - Ejemplo de query+stagger

```
import { trigger, transition, query, style,stagger, animate } from '@angular/animations';

animations: [
   trigger('trigger', [
     transition(':enter', [
       query('.efecto', [
         style({opacity: 0, transform: 'translateY(-100px)'}),
         stagger(30, [
           animate('500ms ease-out',
           style({ opacity: 1, transform: 'none' }))
         ])
       ])
     ]),
     transition(':leave', [
       query('.efecto', [
         style({backgroundColor: 'red'}),
         stagger(30, [
           animate('500ms ease-in',
           style({ opacity: 0, transform: 'none' }))
         ])
       ])
     ])
   ]),
 ]
```

ANGULAR

- Animaciones. Complex Sequences.
  - Ejemplo de query+stagger

```
static contador:number=0;
items:string[]=["Uno","Dos","Tres"];
numeroItems:number;
agregar(){
   BorrarComponent.contador++;
   this.items.push("Nuevo" + BorrarComponent.contador);
}
eliminar(){
   this.items.pop();
}
```